

UNIVERSIDAD MESOAMERICANA

Facultad de Ingeniería · Sede Quetzaltenango
Ingeniería en Sistemas · Sexto semestre · Sección 6D

GameVault

Catálogo de videojuegos con Docker Compose

NGINX

FLASK

POSTGRESQL

REDIS

COMPOSE

Estudiante: Denilson Yair Díaz López

Carné: 202308106

Curso: Redes II

Docente: Sergio Torres

Fecha de entrega: 2 de octubre de 2026

Manual de instalación, arquitectura, pruebas y defensa

1. Resumen del proyecto

GameVault es un catálogo web para registrar y descubrir videojuegos. Centraliza títulos, géneros, plataformas, fechas de lanzamiento, precios y existencias. El usuario puede realizar un CRUD completo, combinar filtros, destacar recomendaciones y guardar favoritos temporales.

Cumplimiento del laboratorio

Requisito	Implementación en GameVault
Cinco servicios	nginx, catalog-api, interactions-api, db y cache
Único puerto público	Solo NGINX publica APP_PORT:80
CRUD	Crear, consultar, editar y eliminar videojuegos
Búsqueda/filtro	Texto, género, plataforma, estado y destacados
Función complementaria	Favoritos y contador de visitas en Redis
Característica distintiva	Clasificación nuevo/clásico/próximo y destacados
Persistencia	Volumen nombrado para PostgreSQL
Datos temporales	Redis sin RDB ni AOF, pérdida controlada
Reproducibilidad	Dockerfiles, Compose, .env.example, scripts y documentación

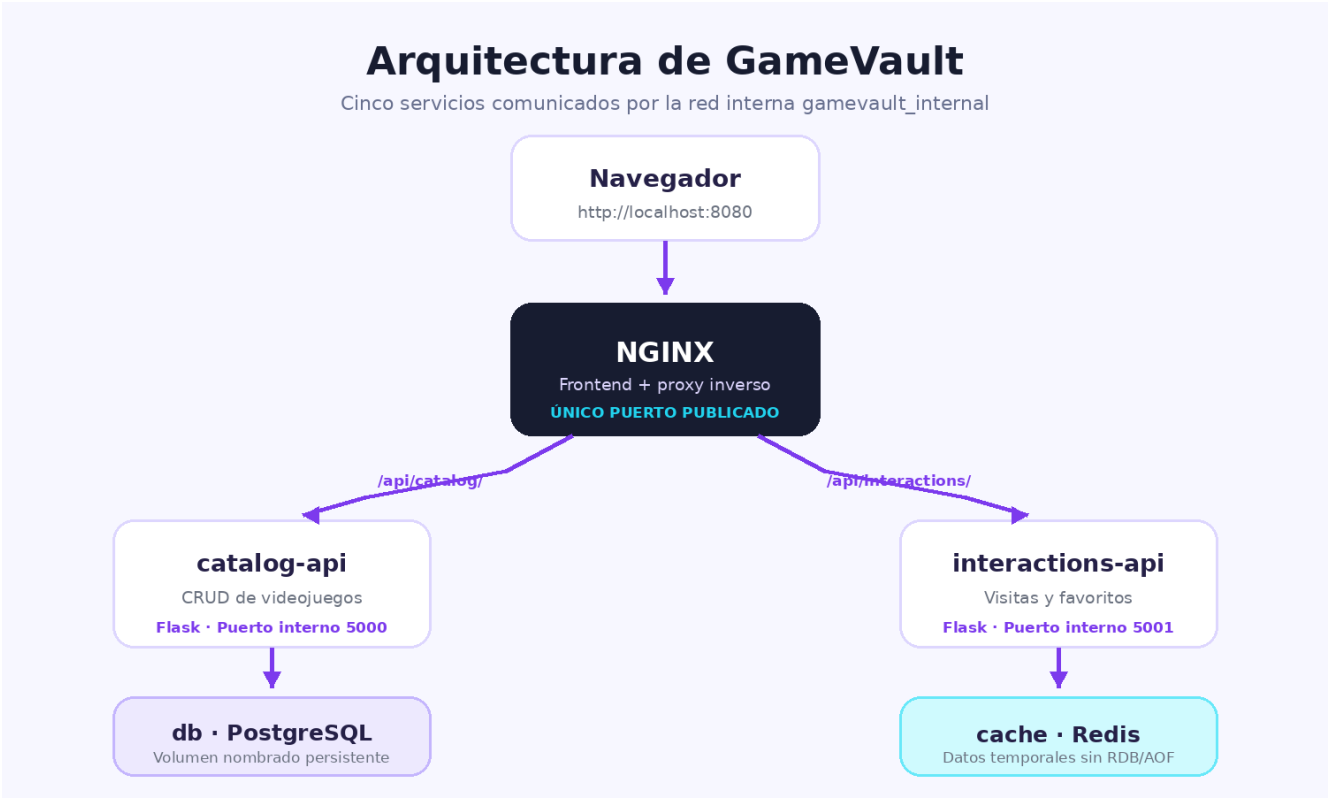
Identidad visual

La interfaz usa una identidad oscura con acentos violeta y cian inspirada en plataformas de videojuegos. El diseño es responsive y contiene jerarquía clara, portadas generadas con CSS, métricas, modales de edición y mensajes de estado.

El proyecto está listo para incorporar los nombres de dos integrantes adicionales si el equipo lo requiere. Esos espacios aparecen señalados en README.md.

2. Arquitectura de servicios

Todas las solicitudes entran por NGINX. El proxy decide si debe servir archivos del frontend o dirigir la petición a una de las API. Las API y bases de datos permanecen aisladas dentro de la red de Docker Compose.



Servicio	Responsabilidad	Comunicación
nginx	Frontend y proxy inverso	Único puerto al host
catalog-api	CRUD, filtros y validaciones	db:5432
interactions-api	Visitas y favoritos	cache:6379
db	Videojuegos permanentes	Volumen postgres_data
cache	Datos temporales	RDB/AOF desactivados

3. Modelo de datos y funcionalidades

Modelo de datos de GameVault

PostgreSQL conserva videojuegos; Redis administra interacciones temporales

games		POSTGRESQL
id	SERIAL · PRIMARY KEY	
title	VARCHAR(120) · NOT NULL	
description	VARCHAR(700) · NOT NULL	
genre	VARCHAR(60) · INDEX	
platform	VARCHAR(60) · INDEX	
release_date	DATE · INDEX	
price	NUMERIC(10,2) · CHECK 0..1000	
stock	INTEGER · CHECK 0..10000	
developer	VARCHAR(120)	
featured	BOOLEAN · DEFAULT FALSE	
created_at / updated_at	TIMESTAMP	

Redis · visitas

gamevault:visits

INTEGER temporal

Se incrementa al abrir la aplicación.

Redis · favoritos

game:<id>:favoritos

SET de identificadores de navegador

Permite agregar o retirar favoritos.

Se borra al reiniciar Redis.

Funciones principales

- Registrar videojuegos con título, descripción, género, plataforma, lanzamiento, precio, existencias, desarrollador y estado destacado.
- Consultar el catálogo y filtrar por texto, género, plataforma o estado.
- Editar todos los campos de un videojuego existente y eliminarlo mediante confirmación.
- Clasificar automáticamente cada título como nuevo, clásico o próximo lanzamiento.
- Marcar favoritos por navegador y observar visitas globales con Redis.

Capas de validación

El navegador valida campos obligatorios y rangos. Catalog API vuelve a validar géneros, plataformas, fechas, precio y stock. PostgreSQL aplica tipos, NOT NULL y CHECK. De esta manera, una solicitud directa tampoco puede insertar datos inválidos.

4. Instalación reproducible

Requisitos: Docker Desktop o Docker Engine, Docker Compose v2, conexión a Internet durante la primera construcción y el puerto 8080 disponible.

Windows PowerShell

```
Copy-Item .env.example .env
```

```
docker compose config
```

```
docker compose up -d --build
```

```
docker compose ps
```

Abra <http://localhost:8080> en el navegador.

Variables principales

Variable	Valor inicial	Función
APP_PORT	8080	Puerto público de NGINX
POSTGRES_DB	gamevault	Nombre de base
POSTGRES_USER	gamevault_user	Usuario de base
POSTGRES_PASSWORD	CambieEstaClave2026	Credencial de práctica
CATALOG_API_PORT	5000	Puerto interno CRUD
INTERACTIONS_API_PORT	5001	Puerto interno interacciones
REDIS_PORT	6379	Puerto interno de Redis

Para detener sin borrar datos use `docker compose down`. Use `docker compose down -v` solamente cuando desee eliminar también PostgreSQL.

5. API y rutas públicas

NGINX conserva un único origen para frontend y API. Las rutas del catálogo usan el prefijo `/api/catalog/` y las interacciones usan `/api/interactions/`.

Catalog API

Método	Ruta	Resultado
GET	<code>/api/catalog/health</code>	Estado de API y PostgreSQL
GET	<code>/api/catalog/games</code>	Lista y filtros
GET	<code>/api/catalog/games/options</code>	Géneros y plataformas
GET	<code>/api/catalog/games/:id</code>	Detalle de videojuego
POST	<code>/api/catalog/games</code>	Crear videojuego
PUT	<code>/api/catalog/games/:id</code>	Editar videojuego
DELETE	<code>/api/catalog/games/:id</code>	Eliminar videojuego

Interactions API

Método	Ruta	Resultado
GET	<code>/api/interactions/health</code>	Estado de API y Redis
POST	<code>/api/interactions/visits</code>	Incrementar visitas
GET	<code>/api/interactions/stats</code>	Totales temporales
GET	<code>/api/interactions/games/favorites</code>	Conteo por videojuego
GET/POST	<code>/api/interactions/games/:id/favorite</code>	Consultar o cambiar favorito

Los errores usan códigos HTTP 400, 404, 500 o 503 y mensajes JSON legibles. La interfaz transforma esos mensajes en avisos visibles para el usuario.

6. Pruebas obligatorias

Prueba	Acción	Resultado esperado
Inicio	<code>docker compose ps</code>	Cinco servicios operativos
Acceso	Abrir <code>localhost:8080</code>	Interfaz visible
Comunicación	Consultar ambos <code>/health</code>	PostgreSQL y Redis connected
CRUD	Crear, buscar, editar y eliminar	Cambios correctos
Persistencia	<code>down</code> y <code>up</code> sin <code>-v</code>	Videojuego permanece
Temporal	<code>restart</code> cache	Visitas/favoritos vuelven a cero
Aislamiento	Probar 5432, 6379, 5000 y 5001	Conexión rechazada
Errores	Detener <code>catalog-api</code>	NGINX devuelve 503
Reproducibilidad	Instalar desde ZIP	Arranque sin ajustes ocultos

Persistencia de PostgreSQL

```
docker compose down
docker compose up -d
Start-Sleep -Seconds 8
```

Pérdida controlada de Redis

```
docker compose restart cache
Start-Sleep -Seconds 5
```

Aislamiento desde PowerShell

```
Test-NetConnection localhost -Port 5432
Test-NetConnection localhost -Port 6379
Test-NetConnection localhost -Port 5000
Test-NetConnection localhost -Port 5001
```

La lista detallada de capturas se encuentra en `docs/pruebas-tecnicas.md`.

7. Video y defensa

El guion incluido dura aproximadamente ocho minutos y cubre todos los puntos solicitados: presentación, aplicación, docker compose ps, CRUD, persistencia, Redis, aislamiento e instalación desde cero.

Orden sugerido

Tiempo	Contenido
0:00 - 0:45	Presentación del problema, solución e integrantes
0:45 - 1:45	Arquitectura y cinco servicios
1:45 - 3:30	CRUD completo
3:30 - 4:20	Filtros, estados y videojuego destacado
4:20 - 5:15	Favoritos, visitas y reinicio de Redis
5:15 - 6:15	Persistencia de PostgreSQL
6:15 - 7:00	Aislamiento y manejo de errores
7:00 - 8:00	Instalación reproducible y cierre

Conceptos que cada integrante debe dominar

- Diferencia entre ports y expose.
- Uso de nombres de servicio como DNS interno.
- Función de NGINX como proxy inverso.
- Volumen nombrado y diferencia entre down y down -v.
- Motivo de desactivar RDB y AOF en Redis.
- Validaciones del frontend, API y base de datos.

El archivo docs/guia-defensa.md contiene respuestas desarrolladas y posibles modificaciones solicitadas por el docente.

8. Lista final de entrega

Antes de comprimir y entregar, marque cada punto:

- Completar los nombres de los integrantes en README.md.
- Copiar .env.example como .env y cambiar la contraseña de práctica.
- Ejecutar docker compose config sin errores.
- Confirmar los cinco servicios con docker compose ps.
- Realizar CRUD completo desde el navegador.
- Demostrar persistencia de PostgreSQL sin usar -v.
- Demostrar pérdida temporal al reiniciar Redis.
- Comprobar que solo NGINX publica puerto.
- Guardar las 15 capturas sugeridas en evidencias/pruebas/.
- Grabar el video de 6 a 10 minutos con participación de todo el equipo.
- Probar el ZIP en otra computadora.

Contenido del paquete

Carpeta/archivo	Contenido
docker-compose.yml	Definición de los cinco servicios
nginx/	Dockerfile y proxy inverso
catalog-api/	CRUD Flask y PostgreSQL
interactions-api/	Visitas/favoritos con Redis
db/	Esquema, índices y datos iniciales
frontend/	Interfaz completa responsive
docs/	Diagramas, IA, pruebas, video y defensa
scripts/	Prueba rápida para PowerShell
evidencias/pruebas/	Ubicación de capturas

GameVault cumple la arquitectura obligatoria y ofrece una base clara para la demostración, la defensa individual y una instalación reproducible.